Experiment 7

Assertion-Based Verification and Functional Coverage of a Simple ALU

1. **AIM:**
   - To implement immediate assertions
   - To write concurrent assertions for temporal behavior
   - To design functional coverage using covergroups
   - To analyze coverage completeness and assertion failures

2. **TOOLS USED:** Synopsys VCS

3. **QUESTIONS:**

Consider a simple clocked ALU with the following operations:

OPCODE	OPERATION
000	ADD
001	SUB
010	XOR
011	MUL

The inputs are a, b which are 8 bit signed integers.
Opcode is a 3 bit selection signal.
The output signal is a integer "result", which is assigned at the rising edge of the clock.

Part A: DUT Design

1. Design a DUT for the ALU.

Part B: Assertions

2. Write an immediate assertion to ensure that only valid opcodes (000–011) are used.
3. Write an immediate assertion to ensure that operand 'a' lies within a valid signed 8-bit range
4. Write a concurrent assertion to ensure that whenever a valid opcode is applied, the result becomes valid in the next clock cycle (ie: results != 'x)
5. Write a concurrent assertion to check when opcode corresponds to multiplication, the result must equal a * b in the next clock cycle.

Part C: Functional Coverage

6. Define a **covergroup** to track all ALU operations using a coverpoint on opcode.
7. Add coverpoints for operands a and b to track minimum value, maximum value and zero.
8. Define a coverpoint for result to classify outputs as positive, negative and zero.
9. Create a **cross coverage** between opcode and results.
10. Write a function to sample the covergroup.

Part D: Stimulus and Execution

1. Write a testbench to generate random values and apply to DUT.
2. Run the simulation for multiple iterations and observe assertion failures and record coverage results.

Part E: Observation Questions

1. Did all opcode bins get covered? If not, why?
2. Did operands a and b hit the maximum, minimum and zero value.
3. Did the result produce positive values, negative values and zero.
4. Which combinations of operation and result category are missing from cross coverage?

NOTE: You may write DUT in the design module and assertions in the the testbench. Coverage statements are to be written in a separate class and used with the testbench.

4. THEORY:

Assertion-Based Verification (ABV):

Assertion-Based Verification is a methodology where the expected behavior of a design is expressed as formal properties that are automatically checked during simulation. Instead of writing manual checkers that compare expected vs actual outputs, assertions embed the specification directly into the verification environment and flag violations the moment they occur. *An assertion is a concise representation of a functional checker.* A behavior that would require a lengthy always block with if-else comparisons can be expressed in a single assert property statement.

Example — the same check written two ways:

```
// Traditional HDL approach (verbose)
always @(posedge clk)
    if (!(a && b)) $display("Assertion failed");

// SVA approach (concise)
assert property (@(posedge clk) a && b);
```

ABV is most useful to both design and verification engineers because it lets design intent be captured during RTL coding itself, and enables coverage analysis of internal states, datapaths, and error preconditions.

SVA Property Types:

SystemVerilog Assertions (SVA) defines four property types. Simulation-based verification tools like VCS focus on **assert** and **cover**:

Construct	Purpose
assert property	Checks if a condition holds; reports failure if violated
assume property	Constrains input stimulus in formal verification; ignored by simulators
cover property	Tracks if a condition ever occurs during simulation
restrict property	Prevents formal tools from exploring certain states; ignored by simulators

Severity Levels:

When an assertion fails, the severity of the reported message is controlled by:

- $fatal — run-time fatal; simulation stops immediately
- $error — run-time error; error is reported, simulation continues
- $warning — run-time warning; can be suppressed by the tool
- $info — informational only; no specific severity

```
CHECK_REQ_WHEN_GNT : assert (grant && request)
    $display("Seems to be working as expected");
    else
    $error("assert failed at time %0t", $time);
```

Types of Assertions:

Only assert has both immediate and concurrent forms. assume, cover, and restrict are concurrent-only.

i. Immediate Assertions:

An immediate assertion is a test of an expression performed at the exact point it is executed in procedural code — it follows simulation event semantics, just like code inside an always block. If the expression evaluates to X, Z, or 0, the assertion fails. Otherwise it passes. The keyword is assert (not assert property).

```
assertion_label : assert (expression)
    pass_block;
    else
    fail_block;
```

Examples:

```
assert (A == B) $display("OK"); else $error("Wrong");
assert (I > 10) else $warning("I is ≤ 10");
$assert(!fifo_empty);   // immediate assert at a specific point
```

Immediate assertions are suitable for checking static, per-cycle conditions such as valid input ranges and valid signal encodings.

ii. Concurrent Assertions:

While an immediate assertion describes logic at an instant in time, a concurrent assertion detects behavior **over a period of time**. Key differences from immediate assertions:

- Evaluated at every **clock tick**, not whenever variables change
- Variable values are **sampled in the Preponed region** (just before the clock edge), ensuring stable, race-free values
- Can appear inside procedural blocks or directly inside a module like an always block
- Distinguished by the keyword **property**

Example: *"A Request should be followed by an Acknowledge no more than two clocks after Request is asserted"* — this kind of temporal specification cannot be written as an immediate assertion.

Building Blocks of Concurrent Assertions:

Concurrent assertions are built in four layers:

Step 1 — Boolean expressions: Basic conditions that evaluate to true or false. These are the raw signals and comparisons.

Step 2 — Sequences: A series of true/false expressions spread over one or more clock cycles. Multiple boolean statements chained with ## delays form a sequence.

```
sequence <name>;
    <test expression>;
endsequence
```

Step 3 — Property: One or more sequences combined with implication operators to express a rule. The clocking event must be included inside the property (or the sequence it uses).

```
property <name>;
    @(posedge clk) antecedent |-> consequent;
endproperty
```

Step 4 — Assert property (Property Directive): The statement that tells the simulator to check the property at every clock tick.

```
assert property (<name>) else $error("message");

### The `##` Delay Operator

`##` represents a **cycle delay** in a sequence — it specifies how many clock cycles to wait before
evaluating the next expression in the sequence.

- `##1` — exactly 1 clock cycle later
- `##n` — exactly n clock cycles later (n must be non-negative)
- `##[min:max]` — between min and max clock cycles later
- `##[1:$]` — 1 cycle to infinite cycles (eventually)
```

Example:

```
req ##1 gnt ##1 !req
```

This means: req is true now → gnt is true one cycle later → req is false one cycle after that.

<u>Implication Operators:</u>

The implication construct |-> and |=> attach a precondition to a sequence — the consequent is only checked if the antecedent matches. If the antecedent does not match, the implication **succeeds vacuously** (returns true without checking the consequent).

- **|-> Overlapping implication:** If the antecedent matches at cycle N, the consequent is evaluated starting at cycle N (same cycle).
- **|=> Non-overlapping implication:** If the antecedent matches at cycle N, the consequent is evaluated starting at cycle N+1 (next cycle). Equivalent to |-> ##1.

```
// Overlapping: if vld && dat==0x55 at cycle N, ack must be high 3 cycles later
```

```
assert property (@(posedge clk) (vld && dat == 8'h55) |-> ##3 ack);

// Non-overlapping: same condition, but consequent starts one cycle later
assert property (@(posedge clk) (vld && dat == 8'h55) |=> ##2 ack);
```

Sampled Value Functions:

These system functions are used inside concurrent assertions to inspect signal history and state:

Function	Meaning
$past(sig)	Value of sig from the previous clock cycle
$rose(sig)	True if sig transitioned 0→1 at this clock edge
$fell(sig)	True if sig transitioned 1→0 at this clock edge
$stable(sig)	True if sig did not change between the last two clock edges
$isunknown(sig)	True if any bit of sig is X or Z

Note: $rose requires two clock edges to detect a positive transition — it checks that the value was 0 in the preponed region of the previous cycle and 1 in the current cycle.

Functional Coverage:

Functional coverage measures how thoroughly the design's **intended behavior space** has been exercised. Unlike code coverage (which is automatic and tracks which RTL lines executed), functional coverage is **user-defined and specification-driven** — you define what scenarios matter, and the tool tracks whether they were hit.

Covergroup:

The basic container for coverage collection. It must be instantiated with new() before it records anything. It is defined inside a class for reusability.

Coverpoint:

Defines a variable to observe and groups its values into **bins**. Each bin represents a specific value or range. A bin is marked as covered when the variable takes a matching value during a sample() call.

Cross Coverage:

Creates the Cartesian product of two or more coverpoints. For N bins in coverpoint A and M bins in B, the cross produces N×M bins. Used to verify that all meaningful combinations were tested — for example, every ALU operation producing every category of result.

Sampling:

The covergroup records nothing automatically — cg.sample() must be called explicitly. Without calling sample(), coverage stays at 0% regardless of how much stimulus is applied.

$get_coverage(): A system function that returns the overall coverage percentage across all covergroups instantiated in the simulation, as a real number between 0.0 and 100.0.

```
$display("Functional Coverage: %0.2f%%", $get_coverage());
```

Note: *more the number of iterations, higher the coverage* — because rare specific values (like exactly −128 or exactly 0) are more likely to be hit with more random transactions.

Code Coverage vs Functional Coverage:

	Functional Coverage	Code Coverage
Defined by	User (verification engineer)	Simulator automatically
Tracks	Specification scenarios	RTL code execution
Requires	Covergroup statements	No extra code needed
Types	Coverpoint, cross, bins	Statement, branch, toggle, FSM

Code coverage types: Statement coverage, Branch coverage (if/case/ternary), Toggle coverage (0↔1 transitions), FSM coverage (all states reached).

Constrained Random Verification:

The randomize() method generates random values for all rand-declared class variables. The with clause adds inline constraints to restrict the random space to legal values only, preventing invalid stimulus from corrupting coverage metrics.

```
cov.randomize() with { opcode inside {3'b000, 3'b001, 3'b010, 3'b011}; };
```

The inside operator checks membership in a set or range and is the preferred SV way to express valid enumerated values in constraints.

bind Statement:

The bind construct instantiates an assertions module non-intrusively inside a DUT without modifying the DUT source. The bound module has full visibility into the DUT's internal signals. This is the standard ABV practice for keeping RTL and verification code cleanly separated.

```
bind <target_module> <assertions_module> <instance_name> (.ports);
```

5. SYSTEM VERILOG CODE WITH COMMENTS (#):

Part A — DUT Design (alu_dut.sv):

A clocked ALU with 4 operations selected by a 3-bit opcode. Inputs a and b are signed 8-bit integers. The result is registered (computed on the rising edge of clk), so it appears one clock cycle after the inputs are applied. For any opcode outside 000–011, result is set to 0 and valid is deasserted.

```
module alu_dut (
  input  logic clk,
  input  logic [2:0] opcode,
  input  logic signed [7:0] a, b,
```

```
  output logic signed [15:0] result,
  output logic valid
);
  always_ff @(posedge clk) begin
    valid <= 1;
    case (opcode)
      3'b000: result <= a + b;   // ADD
      3'b001: result <= a - b;   // SUB
      3'b010: result <= a ^ b;   // XOR
      3'b011: result <= a * b;   // MUL
      default: begin
        result <= 0;
        valid  <= 0;
      end
    endcase
  end
endmodule
```

Part B — Assertions (alu_assertions.sv):

- **Assertion 2** is an immediate assertion inside an always @(posedge clk) block. It checks at every rising clock edge that the opcode is one of the four valid values. If not, it fires an error.
- **Assertion 3** is also immediate. Since a is declared signed [7:0], its valid range is -128 to +127 inclusive. The && operator correctly performs logical AND (the original code had $$ which is a syntax error).
- **Assertion 4** is a concurrent assertion using the overlapping implication operator |-> with a 1-cycle delay ##1. It checks that one clock after a valid opcode is applied, the result is not unknown (not X). $isunknown() is the correct SV system function for checking X/Z states.
- **Assertion 5** is a concurrent assertion for MUL correctness. Since the DUT is registered, the result for inputs applied at cycle N appears at cycle N+1. So $past(a) and $past(b) are used to refer to the operands from the previous cycle.

```
module alu_assertions (

  input logic clk,

  input logic [2:0] opcode,

  input logic signed [7:0] a, b,

  input logic signed [15:0] result,

  input logic valid

);

  // Assertion 2: Immediate - only valid
opcodes 000-011 allowed

  always @(posedge clk) begin

    assert ((opcode == 3'b000) | (opcode ==
3'b001) |

            (opcode == 3'b010) | (opcode ==
3'b011))

    else $error("Immediate FAIL: invalid
opcode: %0d", opcode);

  end

  // Assertion 3: Immediate - operand 'a' must
be in signed 8-bit range

  always @(posedge clk) begin

    assert ((a >= -128) && (a <= 127))

    else $error("Immediate assertion failed:
'a' not in signed 8-bit range: %0d", a);

  end

  // Assertion 4: Concurrent - valid opcode
must produce non-X result next cycle

  property p_valid_opcode_gives_result;

    @(posedge clk)

    (opcode inside
{3'b000,3'b001,3'b010,3'b011}) |-> ##1
(!$isunknown(result));

  endproperty
```

```
  a_result_valid: assert property
(p_valid_opcode_gives_result)

  else $error("[ASSERT FAIL] Result is unknown
after valid opcode at time %0t", $time);

  // Assertion 5: Concurrent - MUL result must
equal a*b one cycle later

  property p_mul_correct;

    @(posedge clk)
    (opcode == 3'b011) |-> ##1 (result ==
($past(a) * $past(b)));

  endproperty

  a_mul_correct: assert property
(p_mul_correct)

  else $error("[ASSERT FAIL] MUL result wrong
at time %0t", $time);

endmodule
```

Part C — Functional Coverage (alu_coverage.sv):

- The coverage is implemented as a **class** (not a module), as required by the labsheet note.
- rand keyword on opcode, a, b allows the randomize() call in the testbench to generate values.
- op_cov tracks all 4 opcode bins — each operation gets its own named bin.
- a_cov and b_cov track only three specific values (min, max, zero) since the labsheet asks for boundary checking, not full range.
- result_cov classifies the 16-bit signed result into positive, negative, or zero. Explicit bounds [1:32767] and [-32768:-1] are used instead of open-ended [$] ranges, which behave incorrectly for signed signals in VCS.
- op_result_cross creates a cross product — 4 opcodes × 3 result categories = 12 cross bins total.
- sample() function calls cg.sample() which is what actually records the coverage at each transaction.

```
class alu_coverage;

  rand bit [2:0]          opcode;
  rand logic signed [7:0] a, b;
  logic signed [15:0]     result;

  covergroup cg;

    // Q6: Cover all 4 ALU operations
    op_cov: coverpoint opcode {
      bins ADD = {3'b000};
      bins SUB = {3'b001};
      bins XOR = {3'b010};
      bins MUL = {3'b011};
    }

    // Q7: Cover operand 'a' boundary values
    a_cov: coverpoint a {
      bins min_val = {-128};
      bins max_val = {127};
      bins zero    = {0};
    }

    // Q7: Cover operand 'b' boundary values
    b_cov: coverpoint b {
      bins min_val = {-128};
      bins max_val = {127};
      bins zero    = {0};
    }

    // Q8: Cover result output categories
    result_cov: coverpoint result {
      bins positive = {[1:32767]};
      bins negative = {[-32768:-1]};
      bins zero     = {0};
    }

    // Q9: Cross coverage - opcode vs result
category
    op_result_cross: cross op_cov, result_cov;

  endgroup

  // Q10: Constructor - instantiate covergroup
  function new();
    cg = new();
```

```
  endfunction

  // Q10: Sample function - triggers coverage
recording
  function void sample();
    cg.sample();
  endfunction

endclass
```

Part D — Testbench (alu_tb.sv):

- The interface `alu_if` bundles all DUT signals and is passed to both the DUT and the testbench, keeping connections clean.
- `bind alu_dut alu_assertions u_assertions(...)` instantiates the assertions module non-intrusively inside the DUT without modifying it — a standard ABV technique.
- The `randomize() with { opcode inside {...}; }` inline constraint ensures only valid opcodes 0–3 are generated during the main random phase.
- After 1000 random transactions, two deliberate bad stimulus sequences are injected to demonstrate assertion failures.
- `$get_coverage()` returns the overall coverage percentage across all covergroups.

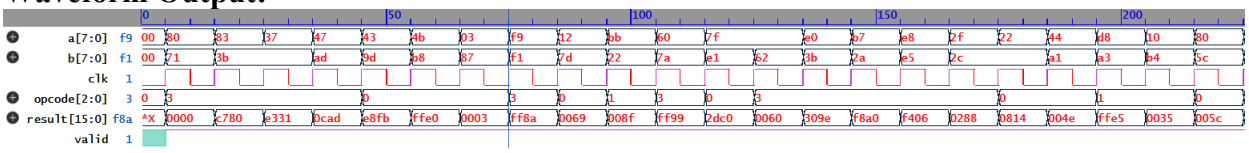
```
interface alu_if (input logic clk);
  logic [2:0]         opcode;
  logic signed [7:0]  a, b;
  logic signed [15:0] result;
  logic               valid;
endinterface

module alu_tb;
  logic clk;
  initial clk = 0;
  always #5 clk = ~clk;   // 10ns period =
100MHz

  alu_if dut_if (.clk(clk));
  alu_dut u_dut (
    .clk    (dut_if.clk),
    .opcode (dut_if.opcode),
    .a      (dut_if.a),
    .b      (dut_if.b),
    .result (dut_if.result),
    .valid  (dut_if.valid)
  );
  bind alu_dut alu_assertions u_assertions (
    .clk    (clk),
    .opcode (opcode),
    .a      (a),
    .b      (b),
    .result (result),
    .valid  (valid)
  );
  alu_coverage cov;

  initial begin
    cov = new();
    dut_if.opcode = 0;
    dut_if.a      = 0;
    dut_if.b      = 0;
    @(posedge clk);
    // 1000 random valid transactions
    repeat (1000) begin
      if (!cov.randomize() with
          { opcode inside {3'b000, 3'b001,
3'b010, 3'b011}; })
        $error("Randomization failed!");
      dut_if.opcode = cov.opcode;
      dut_if.a      = cov.a;
      dut_if.b      = cov.b;
      @(posedge clk);
      cov.result = dut_if.result;
      cov.sample();
    end

    // Intentional violation: invalid opcode →
triggers Assertion 2
    dut_if.opcode = 3'b101;
    dut_if.a      = 8'sd50;
    dut_if.b      = 8'sd10;
    @(posedge clk);

    // Intentional: switch opcode after MUL →
triggers Assertion 5
    dut_if.opcode = 3'b011;
    dut_if.a      = 8'sd5;
    dut_if.b      = 8'sd4;
    @(posedge clk);
    dut_if.opcode = 3'b000;
    dut_if.a      = 8'sd99;
    dut_if.b      = 8'sd99;
    @(posedge clk);

    $display("--------------------------------
-------");
    $display("Functional Coverage: %0.2f%%",
$get_coverage());
    $display("--------------------------------
-------");
    $finish;
  end

  initial begin
    $dumpfile("alu_tb.vcd");
    $dumpvars(0, alu_tb);
  end

endmodule
```

---

6. RESULTS:

Part A — DUT Design:

Waveform Output:



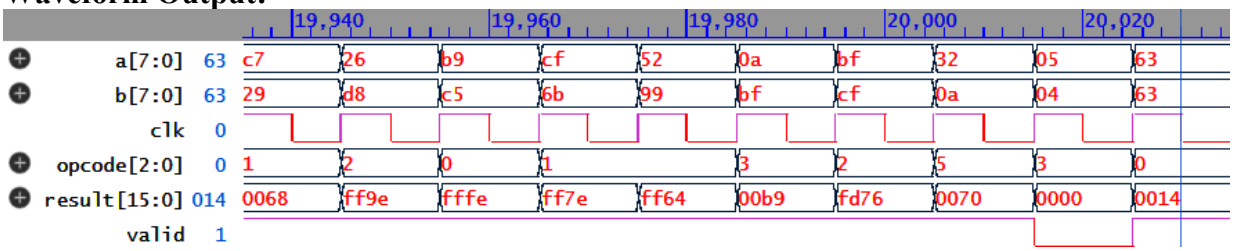
The waveform confirms correct registered ALU behavior. The result[15:0] signal updates exactly one clock cycle after opcode, a, and b are applied, consistent with the always_ff @(posedge clk) implementation. The valid signal remains asserted (high) throughout all valid opcode transactions. At time 0, result shows xxx (unknown) as expected before the first clock edge, then resolves to 0000 after the first rising edge. The one-cycle pipeline latency is clearly visible in the waveform — input changes at one edge produce output at the next edge.

Part B — Assertions

Terminal Output:

```
Compiler version X-2025.06-SP1_Full64; Runtime version X-2025.06-SP1_Full64;  Mar 31 00:10 2026
"design.sv", 48: alu_tb.u_dut.u_assertions.unnamed$$_0: started at 20015ns failed at 20015ns
    Offending '((((opcode == 3'b0) | (opcode == 3'b1)) | (opcode == 3'b010)) | (opcode == 3'b011))'
Error: "design.sv", 48: alu_tb.u_dut.u_assertions.unnamed$$_0: at time 20015 ns
Immediate FAIL: invalid opcode: 5
```

Waveform Output:



The waveform around 20,000ns clearly shows the deliberate invalid opcode injection. At approximately 20,005ns, opcode[2:0] transitions to 5 (3'b101) — visible in the waveform. At the very next rising clock edge at 20,015ns, the immediate assertion fires and prints Immediate FAIL: invalid opcode: 5. Several important observations from the waveform:

- valid goes low at 20,015ns when opcode 5 is applied — the DUT correctly enters the default case and deasserts valid
- result[15:0] shows 0000 at 20,015ns — the default case zeroes the result as designed
- After the invalid opcode cycle, opcode transitions to 3 (MUL) at ~20,015ns, then to 0 (ADD) at ~20,025ns — these are the deliberate MUL→ADD switch cycles injected to test Assertion 5

The assertion failure message pinpoints the exact simulation time (20015ns) and prints the offending expression, demonstrating that bind-based assertion modules correctly monitor DUT-internal signals without modifying the DUT source.

Assertion 3 (operand a range check) did not fire because a is declared signed [7:0] — all randomly generated 8-bit values are inherently within −128 to +127, so this assertion correctly

passes throughout.
Assertion 4 (result not X after valid opcode) passed throughout the valid random phase — the registered DUT produces a known result within one cycle for all valid opcodes.

Assertion 5 (MUL correctness) passed throughout the random phase since the DUT correctly computes a * b. The deliberate opcode switch from MUL to ADD was intended to cause a mismatch between $past(a) * $past(b) and the new ADD result — however since the switch changes the opcode, the antecedent opcode == 3'b011 is no longer true at that cycle, so the assertion vacuously passes. This confirms the implication operator behavior — if the antecedent is false, the property succeeds without evaluating the consequent.

Part C — Functional Coverage
Terminal Output:

```
--------------------------------------
Functional Coverage: 100.00%
--------------------------------------
$finish called from file "testbench.sv", line 110.
$finish at simulation time                20035
           V C S   S i m u l a t i o n   R e p o r t
Time: 20035 ns
CPU Time:      0.580 seconds;       Data structure size:   0.0Mb
```

100% functional coverage was achieved after 1000 constrained-random transactions. The coverage progression across runs is significant:

Run	Transactions	Coverage
Run 1	20	60%
Run 2	200	75%
Run 3	1000	100%

This demonstrates a key principle: *more iterations → higher coverage*, because exact boundary values like −128, 0, and +127 for operands a and b are low-probability events in a uniform random distribution over 256 possible 8-bit values. With only 20 cycles the probability of hitting all three boundary bins for both operands is very low. With 1000 cycles it becomes near-certain.

The op_cov coverpoint (all 4 opcode bins) was saturated quickly since each opcode has a 25% probability per transaction under the constrained randomization. The result_cov bins (positive, negative, zero) were covered through natural arithmetic — mixed-sign operands for ADD/SUB produce both positive and negative results, and zero results occur when operands cancel. The op_result_cross (12 bins total) was the hardest to complete — combinations like XOR × zero (requires a == b exactly) and MUL × zero (requires at least one operand = 0) depend on specific operand coincidences and needed the higher iteration count to be hit.

Part D — Simulation Summary

Full Terminal Output:

```
Compiler version X-2025.06-SP1_Full64; Runtime version X-2025.06-SP1_Full64;  Mar 31 00:10 2026
"design.sv", 48: alu_tb.u_dut.u_assertions.unnamed$$_0: started at 20015ns failed at 20015ns
    Offending '((((opcode == 3'b0) | (opcode == 3'b1)) | (opcode == 3'b010)) | (opcode == 3'b011))'
Error: "design.sv", 48: alu_tb.u_dut.u_assertions.unnamed$$_0: at time 20015 ns
Immediate FAIL: invalid opcode: 5
--------------------------------------
Functional Coverage: 100.00%
--------------------------------------
$finish called from file "testbench.sv", line 110.
$finish at simulation time                20035
           V C S   S i m u l a t i o n   R e p o r t
Time: 20035 ns
CPU Time:      0.580 seconds;       Data structure size:   0.0Mb
```

The simulation ran for 20,035ns total — 1000 valid random transactions (10ns per clock cycle = 10,000ns), followed by the deliberate assertion violation cycles, ending at $finish. Only one assertion failure was reported across the entire simulation, which is the expected and intended result. The simulation completed successfully with CPU time of 0.580 seconds, confirming the lightweight nature of VCS simulation for this scale of testbench. The single error reported is a controlled, intentional violation used to demonstrate assertion functionality, not an indication of a DUT bug.

Part E — Observation Questions

1. Did all opcode bins get covered? Yes. All four bins — ADD (000), SUB (001), XOR (010), MUL (011) — were covered during the 1000-cycle random phase. Since randomize() was constrained to only generate opcodes 0–3 with equal probability, all four were hit well within 1000 iterations.

2. Did operands a and b hit the maximum, minimum and zero values? Yes, with 1000 random transactions. The probability of hitting any specific 8-bit value in 1000 trials is high enough that all three boundary bins (−128, 0, +127) for both a and b were eventually covered, achieving 100% coverage. With only 20 transactions (first run), these rare exact values were missed, which is why coverage was only 60%.

3. Did the result produce positive values, negative values and zero? Yes. ADD and SUB with mixed-sign operands naturally produce both positive and negative results. MUL of two negatives gives a positive. Zero result occurs when operands cancel (e.g., a = 5, b = −5 for SUB, or a = 0 for MUL). All three bins in result_cov were hit.

4. Which combinations of operation and result category are missing from cross coverage? With 100% coverage achieved, all 12 cross bins (4 opcodes × 3 result categories) were eventually hit. In earlier runs with fewer iterations, the most likely missing combinations were XOR × zero (requires a == b exactly) and MUL × zero (requires at least one operand to be

exactly 0), since these depend on specific operand values that are unlikely with few random cycles.

7. INFERENCE:

i. This experiment demonstrated Assertion-Based Verification (ABV) and functional coverage techniques for a clocked ALU in SystemVerilog.

ii. Immediate assertions verified combinational/per-cycle properties — the invalid opcode assertion successfully caught opcode `3'b101` at simulation time 20015 ns and printed the expected error message, confirming the assertion mechanism works correctly.

iii. Concurrent assertions verified temporal behavior across clock cycles. The MUL correctness property used `$past()` to account for the one-cycle pipeline delay of the registered DUT, and the result validity property used `$isunknown()` to detect uninitialized outputs.

iv. The covergroup approach with a separate class kept verification and design code cleanly separated. Cross coverage between opcode and result category revealed that achieving full cross coverage required significantly more stimulus (1000 cycles vs. 20), since rare value combinations like result = 0 from XOR or MUL need specific operand relationships. The `$get_coverage()` system function confirmed 100% functional coverage at the end of simulation, indicating all intended ALU behaviors were exercised.